KAVACH

Supervisor Agent

Deep Technical Explanation + Implementation Flowcharts

A beginner-friendly guide that explains what the agent does, how Python and Gemini work together, how decisions are validated, and how to implement the first prototype.

Purpose	Choose the responsible department, determine priority, prepare a structured action plan, and decide whether the plan can proceed.
Prototype stack	Python, FastAPI, Pydantic, Gemini API, deterministic Python policies, and in-memory sample data.
Decision outcomes	APPROVED_TO_PROCEED, PENDING_APPROVAL, or NEEDS_REVIEW.
Scope	Supervisor Agent only. This document does not design the implementation of other KAVACH agents.

Important: the workflow and code examples in this guide are a proposed implementation based on the Supervisor Agent responsibilities in the KAVACH diagram. Department lists, severity criteria, thresholds and approval rules must be confirmed by the project team.

1. Understand the Supervisor Agent in simple language

Imagine KAVACH receives a complaint: “The dustbin near the main gate is overflowing, garbage is spreading across the walking path, and people are struggling to pass.” The Supervisor Agent acts like a digital operations manager. It determines who should handle the issue, how urgent it is, what needs to happen, and whether the plan is allowed to proceed.

The Duplicate Detection Agent asks whether the same incident has already been reported. The Supervisor Agent asks what operational response is appropriate for the incident. These are different responsibilities.

The five responsibilities in the supplied KAVACH diagram are:

• Analyze incident details.

• Choose a responsible department (for example, Sanitation).

• Set Low, Medium or High priority.

• Decide the required action and prepare an overall plan.

• Determine approval to proceed and return a structured decision.

The Supervisor Agent recommends and validates an operational plan; it does not itself perform physical work. For this prototype, a plan should be a structured object rather than only a paragraph.

2. Proposed prototype technology

Technology	What it does	Why it is used
Python	Implements policies, comparisons, orchestration and validation logic.	Readable, testable functions and modules.
FastAPI	Exposes an HTTP endpoint that accepts a request and returns JSON.	Separates API handling from decision logic.
Pydantic	Validates input/output structure and field types.	Stops malformed or unexpected data entering the policy pipeline.
Gemini API	Interprets natural-language incident details and proposes an action/department.	Flexible language understanding without running a local model.
Python policy modules	Enforce allowed departments, priority rules, action constraints and approval conditions.	AI suggestions cannot bypass mandatory rules.
In-memory repository	Supplies predefined policy data and synthetic incidents during the demo.	Avoids requiring a persistent database in Week 1.

Recommended starting point: implement the agent as a normal Python service. Do not add LangGraph just for appearance. Introduce orchestration tooling only if the actual workflow needs persistent checkpoints, complex retries or human-review cycles.

3. Master technical flowchart

This flowchart shows the proposed end-to-end processing path, including invalid input, AI failure, policy checks, plan validation and approval outcomes.



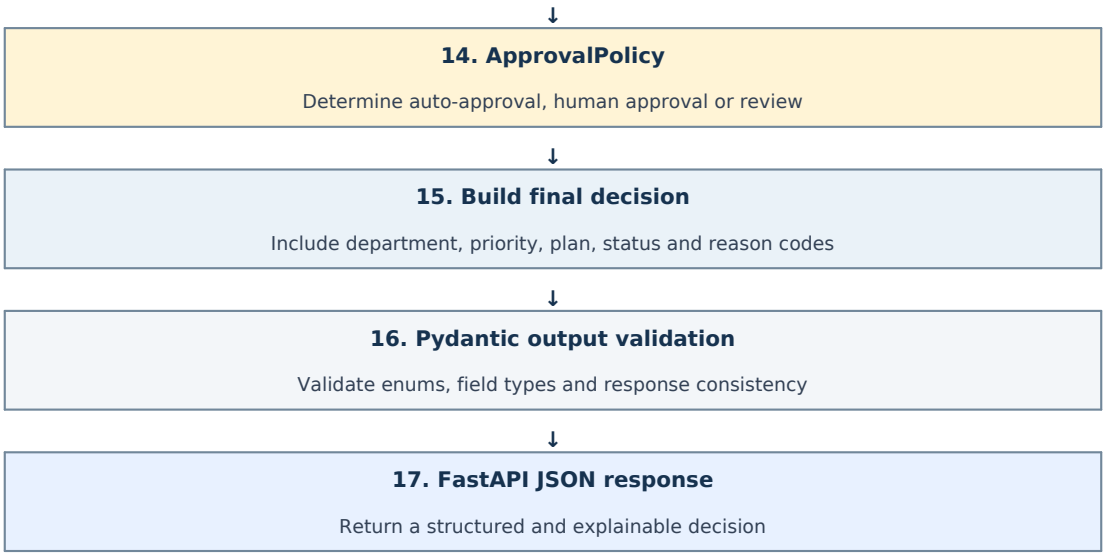


Figure 1. Master sequence. Branches for missing evidence, AI errors, invalid proposals and approval are expanded below.

4. Detailed branching flowchart: failure, policy and approval



APPROVED_TO_PROCEED	PENDING_APPROVAL	NEEDS_REVIEW
Policy and plan checks passed; automatic approval is explicitly permitted.	Plan is valid but a human decision is required before proceeding.	Evidence, department, priority, AI response or plan is insufficient/conflicting/invalid.

5. Step-by-step explanation of the flowchart

Step 1 — Receive a structured incident

The request should contain an incident ID, description, category, available location, evidence-completeness information and relevant severity indicators. The Supervisor Agent should not silently invent facts that were not supplied.

Step 2 — FastAPI endpoint

A route such as POST /api/v1/supervisor/evaluate accepts JSON and calls the service. Keep the route thin: HTTP handling belongs in the route; decisions belong in service and policy modules.

Step 3 — Pydantic validation

Validate required fields and data types before doing any AI work. Invalid structure should result in a clear validation error. Distinguish an absent risk indicator from an explicit false value.

Step 4 — SupervisorService

This is the coordinator. It calls context building, evidence checks, the Gemini client, policy modules, plan construction and output validation in the correct order.

Step 5 — Build decision context

Normalize category IDs and whitespace; preserve the original description; gather available location and risk evidence; record missing or conflicting values. Do not fabricate missing coordinates, risk indicators or timestamps.

Step 6 — Evidence sufficiency

Define minimum information required for the decision. If a key safety fact is missing or evidence conflicts, return NEEDS_REVIEW rather than forcing a confident result.

Step 7 — Constrained Gemini request

Send only necessary incident data, a predefined department list, the meanings of priority labels, allowed action constraints and a response schema. Ask the model to distinguish observed facts from inference and to report uncertainty.

Step 8 — Gemini API call

Use a configured timeout and handle rate limits, network failures and API errors. Do not treat a timeout as a valid recommendation. Keep API keys in environment variables, never source control.

Step 9 — Parse and validate the proposal

Parse the response into a typed proposal. Check department, rationale, suggested actions, risk indicators and uncertainty fields. Invalid JSON or unexpected values must not proceed as if valid.

Step 10 — DepartmentPolicy

Check the proposed department against the allowed list and configured routing rules. If a mandatory mapping conflicts with the AI suggestion, enforce the rule or require review. Multi-department or ambiguous cases need an explicit policy.

Step 11 — PriorityPolicy

Apply the approved priority definitions to the available evidence. Gemini may identify indicators, but Python applies the policy. Treat unknown values as unknown; do not automatically interpret missing evidence as low risk.

Step 12 — ActionPlanBuilder

Build a structured plan with an objective, required actions, rationale, constraints and any relevant resources or conditions. Use approved templates and permit incident-specific suggestions only within defined constraints.

Step 13 — Plan validation

Check required fields, allowed action types, department compatibility and policy constraints. Reject incomplete or disallowed plans. Keep the reason for rejection.

Step 14 — ApprovalPolicy

Decide whether the case may proceed automatically, must wait for a human, or requires review. The AI cannot grant its own approval. A prototype may simulate review but must not claim that a real person approved it.

Step 15 — Build final decision

Create one typed decision object containing department, priority, action plan, approval status, review reasons and reason codes. Keep decisions explainable.

Step 16 — Validate output

Validate enums, score/status semantics if any, required fields and cross-field consistency. For example, a response cannot say approval is pending while also claiming it is approved to proceed.

Step 17 — Return JSON

FastAPI returns the structured decision. The Supervisor Agent does not carry out physical work or decide individual worker assignments as part of this design.

6. Example input contract

The following is an illustrative request, not a finalized project schema.

```
{
  "incident_id": "INC-104",
  "description": "Dustbin overflowing near the main gate; waste is blocking part of the path.",
  "category": "WASTE_OVERFLOW",
  "location": {
    "site_id": "SITE-01",
    "zone": "MAIN_GATE"
  },
  "severity_indicators": {
    "walking_path_obstructed": true,
    "immediate_safety_risk": null
  },
  "evidence_complete": true
}
```

Notice that immediate_safety_risk is null. This means the request has not established whether that condition is present or absent. The policy must not silently convert null to false.

Illustrative Pydantic request model

```
from pydantic import BaseModel
from typing import Optional

class SeverityIndicators(BaseModel):
    walking_path_obstructed: Optional[bool] = None
    immediate_safety_risk: Optional[bool] = None

class LocationInfo(BaseModel):
    site_id: Optional[str] = None
    zone: Optional[str] = None

class SupervisorRequest(BaseModel):
    incident_id: str
    description: str
    category: str
    location: LocationInfo
    severity_indicators: SeverityIndicators
    evidence_complete: bool
```

7. Example AI proposal and policy checks

Illustrative Gemini proposal

```
{
  "department": "SANITATION",
  "reason": "The complaint concerns overflowing waste.",
  "suggested_actions": [
    "Remove overflowing waste",
    "Clean the affected area",
    "Check path access"
  ],
  "risk_indicators": ["WALKING_PATH_OBSTRUCTED"],
  "uncertainties": ["Immediate safety risk was not established"]
}
```

The proposal is not the final decision. Python must check whether SANITATION is allowed, apply the project's priority policy, validate the proposed actions, and evaluate approval requirements.

Example department policy

```
ALLOWED_DEPARTMENTS = {
    "SANITATION",
    "MAINTENANCE",
    "ELECTRICAL",
    "WATER_SUPPLY",
}

DEPARTMENT_RULES = {
    "WASTE_OVERFLOW": "SANITATION",
    "BROKEN_STREETLIGHT": "ELECTRICAL",
    "BROKEN_PIPE": "WATER_SUPPLY",
    "DAMAGED_BENCH": "MAINTENANCE",
}
```

These mappings are teaching examples and must be confirmed by KAVACH's team. Real policies may allow exceptions or more than one department.

8. Priority policy: how to avoid unsafe guessing

Priority labels need definitions agreed by the project team. An illustrative policy might define LOW as routine and limited impact, MEDIUM as meaningful disruption needing timely action, and HIGH as a serious safety concern or another approved urgent condition.

Do not rely on an oversimplified rule such as “if no risk is reported, priority is LOW.” Missing risk information is not proof that no risk exists.

Illustrative policy outline

```
def determine_priority(indicators, evidence_complete):
    if indicators.immediate_safety_risk is True:
        return "HIGH"

    if evidence_complete is False:
        return "NEEDS_REVIEW"

    if indicators.walking_path_obstructed is True:
        return "MEDIUM"

    if indicators.immediate_safety_risk is None:
        return "NEEDS_REVIEW"

    return "LOW"
```

This is still only a simplified teaching example. Before using it, define which evidence is required for each category and what to do when indicators conflict. Do not treat this snippet as an approved real-world safety policy.

9. Action plan structure and approval outcomes

Represent a plan as fields so Python can validate it. A useful initial contract contains incident_id, department, priority, objective, required_actions, constraints, justification and approval_status.

```
{
  "incident_id": "INC-104",
  "department": "SANITATION",
  "priority": "MEDIUM",
  "objective": "Restore safe access near the main gate",
  "required_actions": [
    "Remove overflowing waste",
    "Clean the affected area",
    "Check whether the walking path is clear"
  ],
  "constraints": [
    "Escalate if a new safety hazard is found"
  ],
  "justification": "Reported waste is affecting path access.",
  "approval_status": "PENDING_VALIDATION"
}
```

Use three distinct outcomes:

- APPROVED_TO_PROCEED: all required checks pass and policy explicitly allows the plan to proceed.
- PENDING_APPROVAL: the plan is valid but awaits a human decision.
- NEEDS_REVIEW: evidence, routing, priority or plan is uncertain, contradictory or invalid.

These statuses should be mutually consistent. The AI does not decide that its own output is approved; a Python policy function sets the final status.

10. Suggested code layout

```
app/
  main.py
  api/supervisor_routes.py
  schemas/supervisor_request.py
  schemas/supervisor_response.py
  schemas/action_plan.py
  services/supervisor_service.py
  services/incident_context_builder.py
  services/action_plan_builder.py
```

```
  ai/gemini_client.py
  ai/supervisor_prompt.py
  policies/department_policy.py
  policies/priority_policy.py
  policies/approval_policy.py
  policies/action_policy.py
  validators/proposal_validator.py
  validators/plan_validator.py
  repositories/in_memory_policy_repository.py

tests/
  test_supervisor_endpoint.py
  test_department_policy.py
  test_priority_policy.py
  test_action_plan.py
  test_approval_policy.py
  test_gemini_failure.py
  test_invalid_proposals.py
```

11. Testing and acceptance criteria

Scenario	Expected behavior
Clear waste-overflow complaint	Propose sanitation; produce a structured plan; apply configured priority and approval rules.
Gemini suggests an unknown department	Reject the proposal or return NEEDS_REVIEW.
Missing safety evidence	Do not assume LOW risk; apply the evidence policy and potentially request review.
Gemini timeout or rate limit	Controlled failure; no fabricated proposal and no automatic approval.
Malformed JSON from Gemini	Reject invalid output; no policy bypass.
Plan contains a disallowed action	Plan validation fails; return review or controlled rejection.
Human approval required	Return PENDING_APPROVAL; do not claim approval has happened.
Valid routine case allowed for automatic approval	APPROVED_TO_PROCEED only if all configured conditions pass.
Conflicting department or severity evidence	Return NEEDS_REVIEW with reason codes.

Prototype testing strategy

- Unit-test Python policy modules without calling Gemini.
- Use a mock Gemini client for stable automated tests and simulated failures.
- Run a small number of separate API integration tests with the real API.
- Keep API keys in environment variables and exclude .env from Git.
- Check output consistency, not only whether a response was produced.

12. Definition of done for Week 1

- A FastAPI endpoint accepts a validated incident request.
- Gemini proposals are parsed and validated before policy processing.
- Department, priority, action-plan and approval policies are separate modules.
- Missing and conflicting evidence are represented explicitly.
- AI/API failures never result in a fabricated approval.
- The response contains department, priority, action plan, approval status and reason codes.
- Automated tests cover normal, invalid, missing-evidence and API-failure scenarios.
- The prototype runs without a frontend or persistent database.
- The agent does not perform physical work or select individual workers.

Final summary

FastAPI receives the incident. Pydantic validates it. Python prepares the decision context. Gemini proposes an interpretation and actions. Python policy modules validate the department, determine priority, build/check the plan and enforce approval rules. The final response is validated and returned as JSON. Uncertainty and failure are explicit outcomes—not reasons to guess.